TECHNICAL DEVELOPMENT REPORT · PROFESSIONAL PORTFOLIO

FAST BUY WAVE

Full-stack E-commerce Platform · Solo Development

Angular 18.2.13 + SSR & my-lib-inside · Spring Boot 3.4 / Java 21 · Python 3.12.4 · MySQL 8 · Podman

767	131K+	~90	3
FILES	LINES OF CODE	COMMITS	CONTAINERS

AUTHOR:	Popa Madalina Mariana
PROJECT:	Fast Buy Wave — Full-stack E-commerce
PERIOD:	2025 – 2026
PROJECT TYPE:	Individual Development (Sole Developer)
CLASSIFICATION:	Professional Portfolio — Public Repository

Document produced for portfolio demonstration.

PROJECT OVERVIEW

Note: The repository is public and viewable. However, all sensitive information (OAuth2 credentials, passwords, ECDSA keys, database credentials, network hosts, domain addresses, and any other critical configuration) has been removed or is managed via environment variables and external files to ensure system security. This document serves as a technical guide to illustrate the architectural choices, implemented optimizations, and technical results achieved during development iterations.

Fast Buy Wave is a production-grade full-stack e-commerce platform independently developed by Popa Madalina Mariana (2025–2026). The project features a distributed and decoupled architecture orchestrated via Podman, integrating a custom Angular 18 UI library (my-lib-inside) published as a local npm package, a Spring Boot 3.4.4 modular backend, a dedicated Python server for asynchronous email management, and MySQL 8 as the relational database.

767 PROJECT FILES	**131K+** LINES OF CODE	**~90** GIT COMMITS	**51** TEST FILES	**3** PODMAN CONTAINERS
18 JPA ENTITIES	**19** REST CONTROLLERS	**25** ANGULAR SERVICES	**3** LOCK RECOVERY TYPES	**1** SOLE DEVELOPER

TECHNOLOGY STACK

FRONTEND · Angular 18 Ecosystem

Angular 18	SSR / Express	RxJS 7.8	TypeScript 5.5	Ang. Material
PrimeNG 17	STOMP / WS	Chart.js 4	Bootstrap 5.3	FontAwesome 6
SweetAlert2	OpenLayers 10	TypeDoc 0.28	Karma/Jasmine	my-lib-inside

BACKEND · Spring Boot 3.4 / Java 21 Ecosystem

Spring Boot 3.4	Spring Security	Spring Data JPA	Spring Batch	Spring Retry
Spring WebSocket	Hibernate ORM	Liquibase	MapStruct 1.5	JWT / ECDSA
OAuth2 Google	Stripe SDK	PayPal SDK	Springdoc 2.8	Log4j2
MySQL 8	H2 (test)	Jackson 2		

INFRASTRUCTURE & TOOLING

Podman | Python / aiosmtpd | Gmail OAuth2 API | Mermaid Graphs | pdoc

DISTRIBUTED SYSTEM ARCHITECTURE

The system adopts a distributed and decoupled architecture orchestrated by Podman. Each service can be updated or restarted independently without impacting the others:

1. Angular 18 Frontend + my-lib-inside (SSR on Express)

The custom my-lib-inside UI library is compiled and integrated as a local dependency. It provides pure presentational components (Navbar, Sidebar, Modal, Card, Carousel, Pagination, Form Layout, Table Layout, Tree Layout, Cart Drawer, Empty State, Spinner, Breadcrumb, Footer) with a dynamic input system (15+ types: text, email, password, select, checkbox, date, file, range, etc.) governed by a form engine configurable via TypeScript interfaces. The host application manages business logic, dynamic routing, and state management.

2. Spring Boot 3.4.4 Backend — Modular Monolith

19 REST controllers, 18 JPA entities, and a service layer structured with separate interfaces and implementations. Supports multi-profile configuration (dev/prod/staging) and secure external cryptographic key management.

3. Dedicated Python Server — Asynchronous Email Management

A sophisticated SMTP intermediary relay composed of two synergistic subsystems: an SMTP Intermediary Server injecting anti-reply protection headers (Reply-To masking, Auto-Submitted, X-Block-Replies) with a 30-day volatile forwarding registry; and a Gmail API Bot via continuous OAuth2-authenticated polling (60-second interval) providing automated HTML template responses. Designed to prevent infinite reply loops and protect the original sender from accidental replies.

MySQL 8 serves as the relational database with schema migration managed by Liquibase, fully configured and documented for future migrations.

BACKEND ARCHITECTURE — SPRING BOOT 3.4.4

Security

JWT using ECDSA signatures (Elliptic Curve Digital Signature Algorithm) on the NIST P-256 curve (ES256), with external PEM public/private keys. Dual authentication strategy: traditional (BCrypt email/password) and Google OAuth2. Persistent Database Refresh Tokens with explicit revocation and configurable expiration (3 days).

Complex Data Modeling — Recursive Categories

Self-referencing entities with parent/child relationships, unique constraints on (name, parent_id) to prevent branch duplicates, and @JsonIdentityInfo for recursion-safe serialization. Includes recursive methods for saving, updating, and toggling entire sub-trees.

Complex Data Modeling — Nested Comments

Materialized Path pattern implementation (treePath e.g., "1.5.12", depthLevel) with custom-logic denormalized counters (directRepliesCount, totalRepliesCount). Hierarchical queries optimized via LIKE 'rootId.%' to fetch entire threads in a single call. Recursive soft delete with state propagation to all descendants.

■ Real-time Capabilities

WebSocket integration via STOMP and SockJS for live notifications (comments, replies, edits, deletions), ensuring reactive UI updates without page refreshes.

■ Persistence

JPA/Hibernate with Criteria API for dynamic multi-field filtering and lightweight full-text search with title tokenization. Automated recursive soft delete across correlated entities (listings → products → images).

■ Batch Processing

Spring Boot Batch configured for scheduled jobs (emails, reporting), featuring a custom DynamicTableItemReader capable of processing configurable database tables via REST parameters, with a fully documented structure in the README.

■ Payment Gateway

Stripe and PayPal Checkout integration at a simulated integration level with production-ready architecture, DTOs, and services ready for end-to-end live credentials.

■ DevOps & Logging

Multi-profile configuration (dev/prod/staging) and secure external cryptographic key management. Log4j2 RollingFile strategy (time/size-based rotation at 10MB) with segregated log channels: a global application.log for full traceability and a warnings.log utilizing ThresholdFilter for critical error isolation.

DEVELOPMENT PHASES — 9 ITERATIONS

#01

Setup & Architectural Foundations

Monorepo Bootstrap · Angular 18.2.16 · Shared Library · Core Layout

Angular 18 · Monorepo · my-lib-inside · npm workspaces

■ Monorepo & Workspace

Monorepo configuration with two separate workspaces (angular_fe and my-lib), coordinated build pipeline, and a structured .gitignore to exclude artifacts, logs, environment variables, and IDE configurations.

■ Shared Library Bootstrap

First version of the my-lib-inside library featuring a standalone component architecture, export patterns via public-api.ts, and integration as a local dependency (file:../my-lib/dist/my-lib-inside) within the main workspace.

#02 Dual Security & Dashboard Orchestration

Google OAuth2 · JWT/ECDSA · Dynamic Routing · SSR Hydration

OAuth2 · JWT/ECDSA · Spring Security · JPA Bidirectional · SSR Hydration

■ Dual Authentication

Parallel implementation of Google OAuth2.0 and traditional JWT flow with ECDSA signature on NIST P-256 curve (ES256) using Nimbus JOSE + BouncyCastle. User entity refactoring with clear separation between public and sensitive data (Jackson mixin @JsonProperty(access = WRITE_ONLY) for password).

■ Dynamic Form Engine

Registration flow completion using the library's dynamic form engine: FormFieldConfig, FormConfig, FormButton interfaces with 15+ input types dynamically mapped via input-component-map.ts.

■ Dashboard Orchestration

Dynamic component orchestration service with context-aware routing (category + source in route parameters via RouteManagerService); SSR hydration managed with manual control using isPlatformBrowser() to prevent double-rendering.

■ JPA Data Model

Advertisement entity with bidirectional @OneToMany/@ManyToOne relationships; One-to-One Profile-Advertisement mapping; recursive category serialization resolved using @JsonIdentityInfo (instead of @JsonManagedReference/@JsonBackReference) to prevent loops.

#03 Performance, JPA Specifications & Payments

Optimistic & Pessimistic Locking · Spring Retry · Stripe · PayPal

Spring Retry · @Retryable · Criteria API · Stripe SDK · PayPal SDK

■ JWT Lifecycle Management

Complete token lifecycle management: generation with ECDSA signature on NIST P-256 curve (ES256) using Nimbus JOSE + BouncyCastle, refresh token persisted in database with RefreshTokenServiceImpl, configurable expiration (3 days) and explicit revocation. Granular permissions for USER / ADMIN roles.

■ Concurrency & Resilience

Spring Retry enabled via @EnableRetry (RetryConfig) with @Retryable on critical stock and order update methods. Handles three distinct lock exception types: ObjectOptimisticLockingFailureException (via @Version conflict detection), PessimisticLockingFailureException, and CannotAcquireLockException. Exponential backoff configured at 100ms initial delay with multiplier 2, max 3 attempts. Each exception type has a dedicated @Recover fallback that performs one final direct save attempt before throwing OptimisticLockRetryException. All operations run in REQUIRES_NEW propagation to isolate transactions.

■ JPA Specifications / Criteria API

Multi-criteria product search using ProductSpecifications (Criteria API): dynamically combinable filters for type, country, city, price range (min/max), title (full-text tokenization), condition, agency, and date. CategoryServiceImpl with recursive category tree management (self-referencing, unique constraint on name+parent_id).

■ Payment Modules

Modular payment architecture with Stripe and PayPal Checkout integration at mock-up level. DTOs, services, and dependencies fully implemented and production-ready. PaymentMethod, PaymentStatus, OrderStatus enumerations with defined state machine. Awaiting live credentials for end-to-end testing.

#04 Modal Wrapper Pattern & Responsive Design

Standalone Modal Library · Media Queries · OnPush Change Detection

Modal Pattern · OnPush CD · ChangeDetectorRef · Mobile-First · 576–1200px

■ Modal Wrapper Pattern

Standalone modal library with a generic wrapper component accepting dynamic content via ng-template and ComponentFactoryResolver. Supports nested modals, configurable size/position, and keyboard accessibility (ESC to close, focus trapping). Reusable across the entire application without code duplication.

■ Responsive Design

Mobile-first approach with CSS Media Queries (breakpoints at 576px, 768px, 1024px, 1200px). Adaptive layout for navbar, sidebar, breadcrumbs, and product grid. Conditional component rendering based on viewport size.

■ Change Detection Optimization

OnPush Change Detection Strategy implemented across all library components (lib-navbar, lib-sidebar, lib-breadcrumb, lib-footer, lib-spinner, lib-empty-state) to minimize DOM checks and improve SSR performance. Manual triggering via ChangeDetectorRef.markForCheck() only when necessary.

#05 Form Architecture & Design Token System

lib-form-layout · Advanced Reactive Forms · CSS Variables · _tokens.scss

Reactive Forms · RegEx Validators · _tokens.scss · CSS Variables · 4-col Grid

■ Form Layout Wrapper

Migration of advertisements-create (1,550 lines TS), profile-settings (1,986 lines TS), and admin-category to the lib-form-layout wrapper featuring a responsive 4-column grid, CSS variables gradient system, and recursive section support for nested form groups.

■ Advanced Validation

Reactive Forms with ValidatorPattern: RegEx for civic numbers, IBAN, phone numbers, and URLs. Real-time min-length counter with character count display. Blur-event trigger for lazy validation. AgencyValidatorsUtils for conditional agency type logic.

■ SCSS Token System

Centralized _tokens.scss implementation (132 lines FE + 160 lines library) with variables for colors, gradients, shadows, and breakpoints. Consistent midnight-blue/violet palette across all components.

#06 Admin Panel & Reactive State Management

lib-table-layout · lib-tree-list-layout · ReplaySubject · Immutable Set

ReplaySubject(1) · Immutable Set · lib-table-layout · lib-tree-list-layout

■ Admin Layout Components

AdminUsersList (628 lines TS) migrated to lib-table-layout with TableColumn and TableAction interfaces; AdminCategoryList (545 lines TS) migrated to lib-tree-list-layout with depth-based visualization and optimized collapseDescendants/findNode methods.

■ ReplaySubject & Immutability

OnPush deadlock resolution in tree-view using ReplaySubject(1) to guarantee last-value emission to new subscribers; immutable Set for node toggling without state mutation.

■ Advanced User Management

Self-deactivation protection for current user; dual-state logic (ACTIVE/INACTIVE) with admin tooltips; context-aware dynamic header actions; unified additionalNotes field for BE/FE DTO alignment.

#07 Global Design System & API Documentation

Unified Palette · SweetAlert2 · Swagger OpenAPI · WebSocket STOMP

SweetAlert2 · Springdoc 2.8 · WebSocket STOMP · OpenAPI 3 · Dark Theme

■ Unified Design System

Global standardization with midnight-blue/violet palette. Responsive card layout optimized for 480px → 1052px with smooth transitions. Complete SweetAlert2 CSS overrides and Angular Material custom theme for visual consistency.

■ Contact & Appointment System

ContactRequestServiceImpl (558 lines) with PENDING/ACCEPTED/REJECTED state management. AppointmentServiceImpl (436 lines) with proposal workflow (confirm, reject, propose, cancel). Real-time notifications via WebSocket STOMP. Permission-based utilities for context-aware actions based on role: organizer vs requester.

■ Swagger / OpenAPI

SpringDoc OpenAPI 2.8.6 with CustomSwaggerIndexTransformer (683 lines): custom UI with project branding, dark theme, and CORS configuration. Complete documentation for all 19 REST controllers including authentication schemas and example requests/responses.

#08 Security Hardening, Logging & Developer Tooling

Env Variables · TypeDoc · Mermaid Graphs · Log4j2 · Python API Client

TypeDoc · Mermaid · Log4j2 RollingFile · .env.local · aiosmtpd · Gmail API

■ Security Hardening

Complete repository sanitization: migration of hosts, IPs, credentials, and ECDSA keys to environment variables (.env.local). Updated .gitignore. Custom start:fritz script with dotenv-cli for local network testing (FritzBox integration).

■ TypeDoc & Graph Generation

TypeDoc 0.28 with custom theme (928 lines CSS + 512 lines JS). Mermaid generators for Angular FE dependencies (generate-mermaid-graph.mjs, 227 lines) and shared library (generate-lib-graph.mjs, 266 lines). Python generator for Spring Boot visualization (generate_spring_graph.py, 294 lines).

■ Python API Client & Email Server

Python REST client (api_client.py, 721 lines) for automated endpoint testing. Email Python Server: aiosmtpd + Gmail OAuth2 API for auto-reply management (server.py, 510 lines; gmail_auto_reply.py, 256 lines) — anti-reply protection, 30-day forwarding registry, and automated rejection messages.

#09

Podman Orchestration, SSR Fix & Testing Infrastructure

3 Containers · NG0203 DI Fix · JUnit/Mockito · 50 Karma/Jasmine Specs

Podman · NG0203 Fix · JUnit/Mockito · Karma/Jasmine · H2 in-memory

■ Podman Multi-Container Orchestration

Complete Podman-based orchestration with 3 separate containers: Angular FE + Library (multi-stage build with compiled my-lib-inside), Spring Boot BE, and Python Email Server. Optimized .dockerignore (81 rules). docker_instruction.md (154 lines) as operational guide.

■ SSR DI Fix (NG0203)

Angular NG0203 error resolution (InjectionToken not found in SSR context): server bundle unification with forceConsistentCasingInFileNames in tsconfig. Paths resolution to avoid multiple @angular/core instances. PrimeNG 17.18.10 downgrade to fix font corruption in SSR.

■ Testing Infrastructure

50 .spec.ts files with Karma/Jasmine covering components, services, interceptors, and pipes. Mock AuthGoogleService, modern provideRouter API, HttpClientTestingModule. 1 JUnit/Mockito suite (CartServiceImplTest, 415 lines) for CartService: stock validation, authorization, edge cases with H2 in-memory.

■ Python REST Testing

Python test client (test_api.py, 409 lines) for manual backend endpoint validation. authController.http (420 lines) with complete HTTP scenarios for traditional authentication and Google OAuth2 flow.

PROJECT METRICS

Category	Metric	Value
Codebase	Total lines of code (git diff)	131,259
Codebase	Total files in project	767
Codebase	Git commits	~90
Frontend	Angular components (app)	~20
Frontend	Library components (my-lib-inside)	~17
Frontend	Angular services	25
Frontend	Karma/Jasmine spec files	50
Frontend	Dynamic form input types	15+
Backend	JPA entities	18
Backend	REST controllers	19
Backend	Documented endpoints (Swagger)	19 controllers
Backend	JUnit test suites (CartService)	1 (415 lines)

Concurrency	Lock exception types handled (@Retry)	3 (Optimistic, Pessimistic, CannotAcquire)
Concurrency	Max retry attempts (exponential backoff)	3 × (100ms, 200ms, 400ms)
Concurrency	Transaction propagation	REQUIRES_NEW per operation
DevOps	Podman containers	3 (FE+Lib, BE, Python)
DevOps	Development period	2025 – 2026
DevOps	Team	Sole Developer

BUILD & DEPLOYMENT GUIDE

ANGULAR FRONTEND — LOCAL BUILD

The Angular frontend depends on the my-lib-inside custom library, which must be compiled first. The following commands perform a clean production build and launch the SSR server:

Command	Description
`rd /s /q .angular`	Clears the Angular build cache completely.
`rmdir /s /q dist`	Removes the previous dist output folder.
`ng build --configuration=production`	Compiles the Angular app in production mode, bundling the my-lib-inside library as a local dependency.
`node dist/angular_fe/server/server.mjs`	Starts the SSR Express server from the compiled production bundle.

PODMAN CONTAINER MANAGEMENT

Each container is built and managed independently. Images are also available on Docker Hub.

Type	Command	Description
BUILD	`podman build -t fastbuy-frontend .`	Build Angular FE image (includes my-lib-inside library).
BUILD	`podman build -t fastbuy-backend .`	Build Spring Boot backend image.
BUILD	`podman build -t fastbuy-email-server .`	Build Python email server image.
RUN	`podman run -it --rm -p 4200:4200 fastbuy-frontend /bin/bash`	Run frontend container interactively. Auto-removes on exit.
RUN	`podman run -it --rm -p 8080:8080 fastbuy-backend /bin/bash`	Run backend container interactively. Auto-removes on exit.

RUN	`podman run -it --rm -p 8025:8025 fastbuy-email-server /bin/bash`	Run Python email server interactively. Auto-removes on exit.
REMOVE	`podman rmi -f fastbuy-frontend`	Force remove the frontend image.
REMOVE	`podman rmi -f fastbuy-backend`	Force remove the backend image.
REMOVE	`podman rmi -f fastbuy-email-server`	Force remove the email server image.
REMOVE	`podman rmi -f`	Force remove any image by its hash ID.

DOCKERFILE ANALYSIS — 3 CONTAINERS

■ Angular Frontend — Dockerfile

Base image: node:20 — official Node.js LTS image.

Strategy: Single-stage build: both my-lib source and angular_fe source are copied into /app, preserving the local dependency path (file:../my-lib/dist/my-lib-inside) as configured in package.json.

Environment generation: Environment files (environment.ts and environment.prod.ts) are generated at build time via RUN printf, injecting backend port (8080), SSR host, and host mapping — keeping sensitive configuration out of the repository.

Entry point: node dist/angular_fe/server/server.mjs — launches the compiled SSR Express server directly, serving Angular with server-side rendering.

Note: Commented lines show the full npm install + ng build pipeline, retained for reference. The active Dockerfile assumes a pre-built dist folder is available, enabling faster container startup in development.

■ Python Email Server — Dockerfile

Base image: python:3.12-slim — lightweight Python image, minimizes final image size.

Strategy: Single-stage build: copies all source files, installs dependencies from requirements.txt via pip, and marks start.sh as executable.

Entry point: python server.py — starts the SMTP intermediary relay directly. An alternative ./start.sh entry point is available via commented line for shell-script-based startup if needed.

Dependencies: aiosmtpd, google-auth, google-api-python-client, requests, python-dotenv, Jinja2, and others as defined in requirements.txt.

■ Spring Boot Backend — Dockerfile

Strategy: Multi-stage build — separates compilation from the runtime image, significantly reducing the final image size.

Stage 1 — Build: maven:3.9-eclipse-temurin-21: copies pom.xml and src/, runs mvn clean package -DskipTests to produce the executable JAR.

Stage 2 — Runtime: eclipse-temurin:21-jre-alpine: a minimal Alpine-based JRE image. Only the compiled app.jar is copied from the build stage. Exposes port 8080.

Entry point: java -jar app.jar — starts the Spring Boot application directly.

Advantage: The multi-stage approach ensures the final image contains only the JRE and the JAR — no Maven, no source code, no build tools — resulting in a lean, secure production image.

TECHNICAL REFLECTIONS

Fast Buy Wave represents an end-to-end software engineering journey undertaken entirely independently: from JPA domain modeling to containerized deployment, from designing a reusable Angular library to optimizing concurrency on InnoDB with optimistic locking (@Version), pessimistic lock handling, and Spring Retry with exponential backoff. Each phase produced concrete solutions to real technical problems, documented in commits and measurable in performance metrics.

The key architectural decisions — Modal Wrapper Pattern, dynamic form engine, ReplaySubject for OnPush state management, centralized SCSS token system, Spring Retry with @Recover — reflect a vision oriented toward maintainability and scalability in real team contexts. The result is a production-grade system with 767 files, 131,000+ lines of code, and an architecture documented at every level.